

CS 7295 GPU HW & SW — Project 2: Parallel Bitonic Sort

Author: *Kyle Nguyen* | **GTID:** 903953383 | **Date:** 2026-06-15

1. Introduction and Problem Overview

This project implements a parallel integer sorting routine in CUDA based on **Bitonic Sort**, a comparison-network algorithm that is particularly well suited to GPU execution. Unlike mergesort, whose later merge stages collapse onto a shrinking number of active threads, bitonic sort performs a fixed, data-independent sequence of compare-exchange operations in which **every element is active at every step**. This regularity eliminates load imbalance and divergent control flow, making the algorithm map cleanly onto the SIMT execution model of the NVIDIA H100.

The work is organized in two parts. **Task 1** is a straightforward global-memory implementation that issues one kernel launch per bitonic sub-stage. **Task 2** optimizes this baseline by introducing a **shared-memory kernel** that fuses the small-stride steps of each merge into a single launch, drastically reducing both the number of kernel launches and the volume of global-memory traffic.

2. Algorithm Background

Bitonic sort builds a sorted array bottom-up. For an array of length $n = 2^m$, the outer loop runs stages $i = 1 \dots \log_2 n$. Stage i turns adjacent sorted runs of length 2^{i-1} into bitonic runs of length 2^i and then merges each into a sorted run of length 2^i . The merge for stage i consists of i sub-steps $j = i - 1 \dots 0$, each performing a compare-exchange between every element k and its partner $k \oplus 2^j$.

The sort direction for a comparator is determined purely by position:

```
asc = ((k & (1 << i)) == 0)
```

so even-indexed chunks of length 2^i sort ascending and odd chunks descending, producing the bitonic input required by the following stage. Because each of the $\frac{1}{2} \log_2 n (\log_2 n + 1)$ sub-steps touches all $n/2$ comparator pairs, total work is $O(n \log^2 n)$ with parallel depth $O(\log^2 n)$. The extra $\log n$ factor over a work-optimal sort is the price paid for perfect regularity and parallelism.

3. Implementation

3.1 Data Preparation and Padding

Bitonic sort requires a power-of-two length. The host routine `host_to_dev()` computes the next power of two with a branchless bit-fill `next_pow2()`, allocates the device buffer, and copies the input. The padded tail is filled **on the device** with `INT_MAX` by the `fill_tail` kernel, so the sentinel values sort to the high end and never disturb the valid data. All allocation and initialization happens inside the timed region, in compliance with the project rules (no static/global pre-initialization).

3.2 Task 1 — Global-Memory Kernel

The baseline kernel `compare_exchange_cuda` assigns one thread per array index k . Each thread computes its partner $p = k \oplus 2^j$ and performs the compare-exchange only when $k < p$, which guarantees each pair is processed exactly once and avoids write races. The host issues one launch per (i, j) sub-step:

```
for i = 1 .. log n
    for j = i-1 .. 0
        compare_exchange_cuda<<<grid, block>>>(arr, i, j, n)
```

This is correct but launch-bound: a 100M-element array needs $\frac{1}{2} \log^2 n \approx 350$ kernel launches, and **every** comparator reads two values from global memory and writes two back.

3.3 Task 2 — Shared-Memory Kernel

The key observation is that when the comparator stride 2^j is smaller than the chunk of elements a block owns, the partner $k \oplus 2^j$ always lands inside the same chunk. Those steps therefore need no cross-block communication and can be executed entirely in **shared memory**.

`bitonic_shared` gives each block a contiguous `TILE`-element chunk. It:

1. **Loads** the chunk into a `__shared__` buffer with a grid-stride loop.
2. **Runs every step** from `j_start` down to `0` locally, with a `__syncthreads()` between steps so all comparators of one step complete before the next reads the data.
3. **Stores** the chunk back to global memory once.

Each thread maps its pair index `t` to the low element of its comparator with `low = (t / stride) * (2*stride) + (t % stride)`, which assigns exactly one comparator per thread and removes the wasted half-warp and the `if (k > p)` divergence present in the global kernel. The sort direction still uses the **global** index `base + low`, preserving correctness across the block boundary.

3.4 Dispatch Logic

`bitonic_sort()` keeps the global kernel for the large-stride steps ($2^j \geq \text{TILE}$, where partners cross block boundaries) and switches to a single `bitonic_shared` launch the moment `j` drops below $\log_2(\text{TILE})$, which finishes all remaining steps `j .. 0` of that stage in one launch. For small inputs (below one tile) the code falls back to the all-global path, keeping the 2K and 10K correctness cases robust.

The effect on launch count is substantial: each stage's inner loop of up to $\log_2(\text{TILE})$ global launches is replaced by **one** shared-memory launch, removing the majority of launches and global round trips at scale.

3.5 Tunable Parameters

`student.h` exposes two compile-time knobs so the configuration can be swept with a recompile:

Macro	Meaning	Effect
BLOCK_DIM	threads per block	occupancy / scheduling granularity
TILE	elements per block in shared kernel	global-traffic reduction vs. shared-mem pressure

Grid-stride load/store loops decouple `TILE` from `BLOCK_DIM` , so each can be tuned independently. Larger `TILE` absorbs more steps per launch (better memory throughput and meps) but raises shared-memory pressure, which lowers occupancy.

4. Performance Evaluation

4.1 Methodology

Test hardware. A PACE-ICE H100 node could not be secured during the evaluation window, so all measurements in this report were taken on a cloud GPU instance — **Lambda Cloud, instance type `gpu_1x_h100_pcie`** (a single NVIDIA H100 PCIe). This is the PCIe variant of the H100; it has the same Hopper SM architecture as the PACE-ICE H100 but lower memory bandwidth (HBM2e, ~2 TB/s vs. HBM3 ~3.35 TB/s) and a lower power/clock envelope than the SXM part, and its host–device link is PCIe rather than the faster interconnect on SXM nodes. Measured throughput and transfer times should therefore be read as a conservative lower bound relative to the PACE-ICE grading hardware.

Correctness is checked across the graded sizes (2K, 10K, 100K, 1M, 10M) via the program's `FUNCTIONAL SUCCESS` output. Throughput is measured at 100M elements; **Achieved Occupancy** and **Memory Throughput** are measured at 10M with NSight Compute:

```
ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active \
    --print-summary per-gpu ./a.out 10000000
ncu --metrics gpu__compute_memory_throughput.avg.pct_of_peak_sustained_elapsed \
    --print-summary per-gpu ./a.out 10000000
```

4.2 Parameter Sweep

The shared-memory tile size `TILE` was swept across {1024, 2048, 4096, 8192} with `BLOCK_DIM = 256` . Throughput was measured at 100M elements (best of three

runs, per the grading protocol) and the NSight metrics at 10M. Results:

TILE	Shared mem/block	Occupancy @10M	Mem Thpt @10M	Best meps @100M	Kernel time @100M	Grade score
1024	4 KB	59.41%	46.05%	281.85	106.46 ms	14.00
2048	8 KB	59.45%	47.74%	285.48	97.09 ms	14.71
4096	16 KB	58.20%	48.39%	283.87	89.58 ms	15.39
8192	32 KB	49.72%	47.89%	162.80	85.01 ms	15.64

4.3 Analysis of the Sweep

Kernel time is the cleanest signal and improves monotonically as `TILE` grows, from 106.5 ms at 1024 down to 85.0 ms at 8192. This is the expected payoff of step fusion: a larger tile absorbs more of each merge's small-stride steps into a single shared-memory launch, eliminating global round trips and kernel-launch overhead. The marginal gain shrinks at each doubling (≈ 9 , 8, and 5 ms), showing diminishing returns as the remaining work becomes dominated by the unavoidable large-stride global passes.

Occupancy holds near 59% through `TILE = 4096` but falls to 49.7% at 8192, where 32 KB of shared memory per block limits the number of resident blocks per SM. **Memory throughput** peaks at `TILE = 4096` (48.4%) and dips slightly at 8192, consistent with the occupancy loss reducing the SM's ability to hide memory latency even though raw global traffic keeps falling.

meps is dominated by D2H transfer noise. The 100M D2H time swings between ~ 205 ms and ~ 460 ms across otherwise-identical runs, and only runs that happened to land a fast D2H reached ~ 282 – 285 meps. The 8192 configuration never drew a fast D2H run in this batch, so its best meps (162.8) reflects transfer-time variance, not a kernel regression — its kernel time is in fact the best of all four. This transfer variance is server-load dependent and is why the grading protocol takes the best of several runs.

Best configuration. `TILE = 4096` is the best balance: it has the highest memory throughput, retains $\sim 58\%$ occupancy, and posts a strong 89.6 ms kernel time with a stable ~ 284 meps. `TILE = 8192` yields a marginally lower kernel

time but sacrifices occupancy and is more exposed to transfer variance. All four configurations passed functional correctness on every graded size.

4.4 Observations Relative to Targets

The measurements above were taken on a Lambda Cloud `gpu_1x_h100_pcie` instance rather than a PACE-ICE H100 node. Achieved Occupancy (~59%) and Memory Throughput (~48%) fall short of the 65% / 75% targets on this hardware. The occupancy figure is architecture-driven and should carry over closely to the PACE-ICE H100, but the memory-throughput percentage is taken against this part's lower HBM2e peak; on the higher-bandwidth SXM H100 the same kernels are expected to land closer to the target. The kernel time already maps to a strong Option-2 score (≈8.9–9.4 of 10), and reducing D2H exposure is the primary remaining lever for the transfer-time component — the PCIe host link on this instance also inflates the H2D/D2H times relative to an SXM node.

5. Profiler Analysis

NSight Compute was run at 10M elements with per-kernel summaries. The aggregate figures in the sweep table are the arithmetic mean across the three kernels; the per-kernel breakdown below explains what drives them.

Achieved Occupancy (`sm__warps_active.avg.pct_of_peak_sustained_active`):

Kernel	Occupancy	Invocations
bitonic_shared	95.14%	24
compare_exchange_cuda	58.63%	91
fill_tail	24.30%	1

Memory Throughput (`gpu__compute_memory_throughput...`):

Kernel	Mem Throughput	Invocations
bitonic_shared	51.06%	24
compare_exchange_cuda	57.59%	66
fill_tail	34.89%	1

The breakdown is the key insight: the **shared-memory kernel itself reaches 95% occupancy**, well above the 65% target. The reported aggregate (~59%) is pulled down by the global-pass kernel (58.6%) and the one-shot `fill_tail` padding kernel (24.3%). Since grading averages across kernels, the global passes — not the shared kernel — are what hold the headline occupancy below target.

Memory access counts

```
( l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum ,  
  l1tex__t_sectors_pipe_lsu_mem_local_op_ld.sum ):
```

- Every launch of either compute kernel issues **2,097,152 global-load sectors** — exactly one full pass over the padded 16M-element (64 MB) array ($16\text{M} \times 4\text{ B}/32\text{ B}$ per sector). Both kernels read the array once per launch; the shared kernel's advantage is therefore **fewer launches** (24 vs. 66–91), since each shared launch fuses several merge steps internally, not fewer sectors per launch.
- **Local-memory loads are 0** for all kernels, confirming no register spilling.

Divergence

(`smsp__thread_inst_executed_per_inst_executed.ratio`): the measured ratio is **32.0** (full warp) for *both* compute kernels, i.e. no warp divergence in either. The `if (k > p) return guard` in the global kernel is resolved by predication rather than a divergent branch, so the shared kernel's one-comparator-per-thread mapping matches — not beats — the baseline on this metric. The shared kernel's gains come from reduced launches and global traffic, not from removing divergence.

6. Discussion of Optimizations and Effectiveness

Optimization	Rationale	Effectiveness
Device-side padding (<code>fill_tail</code>)	avoids host loop and keeps prep on GPU	enables non-power-of-two inputs at negligible cost
One thread per unique comparator	avoids double-processing and races	correctness + halves comparator launches
Shared-memory step fusion	removes per-step global round trips and launches	primary driver: kernel time 106.5 → 85.0 ms as TILE grows

Optimization	Rationale	Effectiveness
Grid-stride load/store	decouples TILE from BLOCK_DIM	enables independent tuning of occupancy vs. traffic
Position-based direction	branch-free, data-independent ordering	uniform control flow, low divergence

7. Conclusion and Future Work

The two-kernel bitonic design satisfies the functional requirements on every graded size and demonstrates a clear optimization path from a launch-bound global baseline to a shared-memory implementation that minimizes global traffic and kernel launches. Sweeping the tile size confirmed step fusion as the primary lever: kernel time fell from 106.5 ms (`TILE = 1024`) to 85.0 ms (`TILE = 8192`), with `TILE = 4096` giving the best overall balance of kernel time (89.6 ms), occupancy (58%), and memory throughput (48%). Occupancy and memory throughput on the test instance (~59% / ~48%) trail the 65% / 75% targets; measured on a Lambda Cloud `gpu_1x_h100_pcie` (H100 PCIe) rather than a PACE-ICE SXM H100, the memory-throughput and transfer figures in particular are expected to improve on the higher-bandwidth grading hardware.

Promising further optimizations include: (1) **dynamic shared memory** with `cudaFuncSetAttribute` to push `TILE` beyond 8192 ints; (2) **vectorized `int4` loads/stores** to raise effective memory bandwidth; (3) processing **multiple elements per thread** to amortize indexing overhead; and (4) an initial fused "build" kernel that sorts each tile from scratch in shared memory, collapsing the earliest stages into a single launch.